

GLaSS: Global Load-aware Scheduling for Spiking Neural Networks

Your N. Here
Your Institution

Second Name
Second Institution

Abstract

[?]Neuromorphic computing offers a promising energy-efficient paradigm for serving Spiking Neural Networks (SNNs). However, deploying SNNs as a cloud service faces significant challenges due to the unique spatio-temporal sparsity and bursty nature of spiking workloads. Traditional static scheduling policies (e.g., Best-Fit, DRF) fail to adapt to runtime workload fluctuations, leading to hardware hotspots and Service Level Agreement (SLA) violations. In this paper, we propose GLaSS, a Global Load-aware SNN Scheduler designed for multi-tenant neuromorphic clusters. GLaSS introduces a two-tier timing architecture that decouples physical execution from global scheduling to handle the high-frequency dynamics of SNNs. We propose a weighted spike-activity load metric and an ROI-Greedy migration strategy that dynamically rebalances workloads based on migration cost-efficiency. Extensive experiments demonstrate that GLaSS reduces load variance by over 70% compared to static baselines and significantly improves system throughput under bursty traffic conditions.

1 Introduction

As Artificial Intelligence (AI) permeates web services, the energy consumption of deep learning models has become a bottleneck. Spiking Neural Networks (SNNs), running on neuromorphic hardware (e.g., Darwin3, Loihi), offer a sustainable alternative by leveraging event-driven computation. We envision a future of *Neuromorphic Computing-as-a-Service* (NCaaS), where heterogeneous SNN tasks—from robotic control to sequential recommendation—share a unified hardware cluster.

However, serving SNNs in a multi-tenant environment presents unique challenges distinct from traditional DNN serving:

1. **Stochastic Burstiness:** Unlike the deterministic compute graph of DNNs, SNN workloads are highly data-dependent. A "quiet" model may suddenly generate

a spike burst, causing instantaneous congestion in the Network-on-Chip (NoC).

2. **Static Binding Inflexibility:** Existing neuromorphic compilers typically employ static mapping strategies (e.g., Best-Fit or Dominant Resource Fairness). Once deployed, tasks are pinned to cores. When multiple co-located tasks burst simultaneously, "Head-of-Line" blocking occurs, degrading tail latency.

To address these issues, we propose **GLaSS (Global Load-aware SNN Scheduler)**. Unlike general-purpose cluster schedulers (e.g., Kubernetes), GLaSS is aware of the neuro-synaptic dynamics.

Our key contributions are:

- We propose a **Two-Tier Timing Architecture** that bridges the gap between millisecond-level physical execution and second-level global scheduling.
- We design a **Accumulated Spike-Activity Load** metric to accurately quantify the computational and communication pressure of SNNs.
- We develop an **ROI-Greedy Migration Strategy** that optimizes the trade-off between load balancing benefits and state migration costs (S_{state}).
- Evaluation shows GLaSS outperforms static baselines, reducing load imbalance by 73% under bursty arrival patterns.

2 Footnotes, Verbatim, and Citations

Footnotes should be placed after punctuation characters, without any spaces between said characters and footnotes, like so.¹ And some embedded literal code may look as follows.

¹Remember that USENIX format stopped using endnotes and is now using regular footnotes.

```
int main(int argc, char *argv[])
{
    return 0;
}
```

Now we’re going to cite somebody. Watch for the cite tag. Here it comes. Arpachi-Dusseau and Arpachi-Dusseau co-authored an excellent OS book, which is also really funny [?], and Waldspurger got into the SIGOPS hall-of-fame due to his seminal paper about resource management in the ESX hypervisor [?].

The tilde character (~) in the tex source means a non-breaking space. This way, your reference will always be attached to the word that preceded it, instead of going to the next line.

And the ‘cite’ package sorts your citations by their numerical order of the corresponding references at the end of the paper, ridding you from the need to notice that, e.g., “Waldspurger” appears after “Arpachi-Dusseau” when sorting references alphabetically [?, ?].

It’d be nice and thoughtful of you to include a suitable link in each and every bibtex entry that you use in your submission, to allow reviewers (and other readers) to easily get to the cited work, as is done in all entries found in the References section of this document.

Now we’re going take a look at Section ??, but not before observing that refs to sections and citations and such are colored and clickable in the PDF because of the packages we’ve included.

3 Related Work

3.1 Maintaining the Integrity of the Specifications

The IEEEtran class file is used to format your paper and style the text. All margins, column widths, line spaces, and text fonts are prescribed; please do not alter them. You may note peculiarities. For example, the head margin measures proportionately more than is customary. This measurement and others are deliberate, using specifications that anticipate your paper as one part of the entire proceedings, and not as an independent document. Please do not revise any of the current designations.

4 Preliminaries

4.1 SNN Service Model

We consider a stream of SNN inference tasks submitted to the cluster. Each task i is characterized by its neuron count N_i , synaptic weight storage M_i , and a time-varying firing rate $\lambda_i(t)$. The state size S_{state} represents the context (membrane

potentials, synaptic weights) that must be transferred during migration.

4.2 Neuromorphic Cluster Resource

The cluster consists of M neuromorphic cards (nodes). Each card provides limited resources: Cores (C), Synaptic Memory (S_{mem}), and Routing Bandwidth (B).

4.3 The Scheduling Problem

The objective is to find a mapping function $\mathcal{M} : \text{Tasks} \rightarrow \text{Cards}$ that evolves over time to minimize the load variance across all cards, thereby minimizing queuing delay and maximizing cluster throughput.

5 SYSTEM DESIGN

The fundamental challenge in scheduling Spiking Neural Networks (SNNs) on a multi-card Compute-in-Memory (CIM) cluster (e.g., Darwin) lies in the *Spatio-Temporal Coupling Calamity*. Unlike ANNs where execution flows are static and data-independent, SNNs generate discrete, highly time-varying, and input-dependent event streams (spikes). Traditional static scheduling formulates this as a Time-dependent Mixed-Integer Non-Linear Programming (MINLP) problem, which is inherently NP-Hard and suffers from state explosion, failing to prevent instantaneous Network-on-Chip (NoC) congestion.

To break this bottleneck, we propose a two-stage Spatio-Temporal Proactive Scheduling (STPS) framework. It shifts the paradigm from *reactive congestion handling* to *proactive traffic shaping*.

5.1 System Architecture Overview

Our framework decouples the MINLP problem into an offline workload profiling stage and an online hierarchical scheduling stage.

- **Offline Profiler:** Transforms the opaque, input-dependent SNN into a Discrete-Time Dynamic Graph (DTDG), extracting domain-invariant spatio-temporal fingerprints (e.g., burstiness, centrality).
- **Online STPS Scheduler:** Utilizes the extracted fingerprints to perform macro-level multi-card dispatching, micro-level spatial mapping, and most importantly, micro-level temporal phase-shifting to avoid NoC collision.

5.2 Offline DTDG Workload Fingerprinting

To eliminate the randomness caused by individual inputs, we feed a test dataset into the SNN simulator to extract the *statistical expectation* of the spike traffic. The SNN workload is modeled as a DTDG sequence $\mathcal{G}_{dynamic} = \{G^{(1)}, \dots, G^{(T_{window})}\}$.

At time step t , the dynamic graph $G^{(t)} = (\mathcal{V}, \mathcal{E}^{(t)}, \mathbf{W}^{(t)})$ models the neural populations $\mathcal{V}$ and their active event flows. The dynamic edge weight $w_{ij}^{(t)}$ from population i to j is decoupled into a 2D vector: NoC Traffic Volume and Compute Trigger (SOPs).

Based on $G^{(t)}$, we extract three critical physical fingerprints to guide the system scheduling:

1. **In-Eigenvector Centrality ($Var(\mathbf{c}^{(t)})$):** Measures the variance of compute hotspots. A high variance indicates severe local PIM array pressure. We use Power Iteration for $O(|\mathcal{E}^{(t)}|)$ fast approximation.
2. **Global Burstiness (β) & Traffic Sequence ($E^{(t)}$):** Defined as $\beta = \max_t(E^{(t)})/\mathbb{E}[E^{(t)}]$. It quantifies the tidal nature of the NoC traffic. $E^{(t)}$ is recorded as a 1D timeline vector for phase-shifting.
3. **Active Connected Components ($\bar{K}$):** Represents the average number of decoupled sub-graphs. A high $\bar{K}$ indicates the model is highly fragmented and asynchronous.

5.3 Online Spatio-Temporal Proactive Scheduler

When a new SNN inference task T_{new} arrives, the online scheduler reads its DTDG fingerprint and executes a 3-stage hierarchical pipeline.

5.3.1 Stage 1: Macro-Card Dispatching

We solve the cross-card isolation constraint by formulating it as a fragmentation-aware and temporal-load-aware bin packing problem.

- **Fragmentation Matching:** If T_{new} is highly cohesive ($\bar{K} \rightarrow 1$), we allocate it to the card with the largest contiguous idle PIM cores. Conversely, if T_{new} is highly decoupled (large $\bar{K}$), we assign it to heavily fragmented cards to maximize cluster utilization.
- **Temporal Load Isolation:** To avoid simultaneous NoC breakdowns across the cluster, we maintain a running average burstiness β_{card} for each card. High- β tasks are strictly dispatched to the card with the lowest β_{card} (i.e., cards running smooth/flat tasks).

5.3.2 Stage 2: Micro-Temporal Phase-Shifting

Once assigned to a target card C_m , the exact injection time of the input data dictates the runtime NoC congestion. Instead of letting tasks run blindly, we employ a sliding-window cross-correlation algorithm to find the optimal clock offset Δt_{start} .

Let $E_m^{(t)}$ be the forecasted background traffic timeline of card C_m , and $E_{new}^{(t)}$ be the fingerprint of the incoming task. The optimal offset minimizes the absolute peak traffic:

$$\Delta t_{start}^* = \arg \min_{\Delta t \in [0, D_{max}]} \max_t (E_m^{(t)} + E_{new}^{(t-\Delta t)}) \quad (1)$$

where D_{max} is the maximum tolerable QoS delay. This transforms a 3D scheduling problem into a 1D convolution operation, executing in microseconds.

Algorithm 1 Temporal Phase-Shifting Dispatch

- 1: **Input:** Background traffic E_m , Task traffic E_{new} , Max delay D_{max} , Bandwidth BW_{max}
 - 2: **Output:** Optimal offset Δt_{start}^*
 - 3: $MinPeak \leftarrow \infty, \Delta t_{start}^* \leftarrow -1$
 - 4: **for** $\Delta t = 0$ to D_{max} **do**
 - 5: $CombinedTraffic \leftarrow E_m + \text{Shift}(E_{new}, \Delta t)$
 - 6: $CurrentPeak \leftarrow \max(CombinedTraffic)$
 - 7: **if** $CurrentPeak \leq BW_{max}$ **and** $CurrentPeak < MinPeak$ **then**
 - 8: $MinPeak \leftarrow CurrentPeak$
 - 9: $\Delta t_{start}^* \leftarrow \Delta t$
 - 10: **end if**
 - 11: **end for**
 - 12: Update card state: $E_m \leftarrow E_m + \text{Shift}(E_{new}, \Delta t_{start}^*)$
 - 13: **return** Δt_{start}^*
-

5.3.3 Stage 3: Micro-Spatial Mapping

Using the default practices of the Darwin compiler and satisfying hardware constraints, the simulator employs the following method. Finally, the logic populations are statically mapped to the PIM cores. We apply **Hotspot Splitting** guided by $Var(\mathbf{c}^{(t)})$. Populations with extreme centrality scores are geographically split across multiple adjacent PIM cores. This trades a minimal spatial overhead for a significant reduction in instantaneous SOP (Synaptic Operations) accumulation bottlenecks, ensuring thermal and structural load balance on the die.

6 Evaluation

To comprehensively evaluate STPS, we construct a cycle-accurate, trace-driven simulation framework modeling a multi-card Compute-in-Memory (CIM) neuromorphic cluster. We aim to answer three core questions:

- **Q1 (Spatial Load Balancing):** How effectively does STPS’s DTDG-guided placement mitigate resource fragmentation?
- **Q2 (Temporal Load Balancing):** Under extreme event-driven bursts, how does temporal phase-shifting flatten the aggregated NoC traffic over time, and how does this translate to SLA adherence?
- **Q3 (Scalability limits & Ablation):** How do the spatial and temporal balancing modules individually contribute to expanding the cluster’s usable capacity under heavy loads?

6.1 Experimental Setup

Hardware Architecture: We simulate a cluster of $M = 4$ neuromorphic cards. Each card implements an 24×24 2D-Mesh NoC connecting 256 neuromorphic cores. The per-link NoC injection bandwidth is constrained to BW_{max} .

Workloads & Traces: We evaluate diverse real-world SNN models compiled via SpikingJelly [?]. The workloads are strictly categorized into two traffic patterns:

1. *Steady-Flat* (e.g., VGG on CIFAR-10): High spatial synaptic requirement but temporally stable firing rates ($\beta \approx 1$).
2. *Sparse-Bursty* (e.g., Spiking-ResNet on DVS128 Gesture): Driven by event-camera streams, exhibiting extreme temporal tidal bursts ($\beta \gg 1$), which severely test temporal load balancing.

Task arrivals follow a continuous Poisson process (λ). Load balancing is achieved continuously at the admission phase of each incoming task.

Baselines: We benchmark STPS against four standard cluster scheduling heuristics adapted for neuromorphic constraints: Round-Robin (**RR**), **BestFit** (spatial-only greedy packing), Dominant Resource Fairness (**DRF**), and Power of Two Choices (**P2C**). The following examples serve merely as baselines for Step A: Macro Multi-Card Task Dispatching. All baselines adopt the same Step C, while Step B represents a feature specific to SNN tasks.

6.2 Q0: Comparison of End-to-End Algorithm Load Balancing Metrics

6.3 Q1: Spatial Load Balancing via Step A

The goal of Step A is to dispatch tasks across the $M = 4$ cards to maintain an equitable distribution of compute/memory pressure, preventing localized fragmentation. We quantify this using the Coefficient of Variation (CV) and Jain’s Fairness Index (JFI) across all 4 cards.

Table ?? (placeholder) reports the spatial metrics under a continuous stream of active tasks. **1) Mitigating "Head-of-Line" Fragmentation:** Since all baselines share the identical Step C micro-mapping, any variance in cluster-wide fragmentation stems entirely from Step A. STPS achieves the most robust spatial equilibrium, reducing CV by 20.1% against RR and 11.2% against BestFit. The fundamental flaw of DRF and BestFit is treating SNN workloads as continuous, splittable scalar blocks (e.g., C cores, M memory). However, SNN tasks are topological graphs. STPS’s Step A extracts the topological connectivity ($\bar{K}$) from the DTDG fingerprint at admission. By accurately dispatching highly cohesive sub-graphs ($\bar{K} \rightarrow 1$) to cards with contiguous idle PIM blocks, STPS fundamentally prevents spatial resource splintering.

2) The Fallacy of Randomized Placement: P2C yields the worst fairness (JFI ≈ 0.67). In CPU clusters, randomized dispatching is statistically viable. However, neuromorphic tasks exhibit extreme structural heterogeneity. P2C blindly collocates fragmented graphs onto the same card, exhausting local NoC routers while leaving adjacent cards underutilized. STPS’s fragmentation-aware matching completely avoids this mismatch.

6.4 Q2: Temporal Flattening via Step B

In event-driven neuromorphic computing, achieving spatial balance (Step A + C) is necessary but insufficient. SNN bursts occur in the microsecond domain. We measure temporal unbalance using the **SLA Violation Rate** (the percentage of clock cycles where a link’s instantaneous injection rate exceeds BW_{max}).

Counter-intuitive Insight: The Illusion of Average Bandwidth. As shown in Fig. ??, STPS achieves a remarkably low SLA violation rate of 5.37%, thoroughly outperforming DRF (7.46%) and BestFit (6.86%). Because the baselines lack Step B, they inject task data immediately upon dispatch. Even if DRF perfectly balances the *spatial average* load across cards, it is completely blind to temporal variance. Consequently, the biological "spike tides" of collocated bursty tasks randomly overlap, shattering the instantaneous BW_{max} limit.

STPS introduces proactive **Temporal Flattening** through Step B. By leveraging the 1D traffic timeline fingerprint $E_{new}^{(t)}$, Step B executes a sliding-window cross-correlation to calculate an optimal clock offset Δt_{start}^* . This phase-shifting intentionally delays the task’s execution onset by a few microseconds, forcing its burst peaks to fall perfectly into the temporal "valleys" of the background traffic. This zero-overhead temporal manipulation functionally acts as a traffic multiplexer, achieving mathematical temporal load balancing without adding any physical NoC bandwidth.

6.5 Q3: Scalability and Ablation Study

To test the system limits and perform an ablation analysis, we scale the concurrent task count from 80 to 200 (Fig. ??).

1) Unlocking Hidden Capacity (The critical role of Step B): In the critical operating range (140–170 tasks), STPS demonstrates maximum superiority. At 160 tasks, STPS maintains a 21.0% violation rate, outperforming Best-Fit (26.3%) by 20.1%. In this zone, physical spatial resources (Cores/Mem) are heavily fragmented. While static baselines trigger massive congestion, STPS safely "squeezes" newly arrived tasks into the remaining *temporal voids* via Step B, delaying cluster saturation significantly longer than purely spatial schedulers.

2) Convergence at Physical Limits: As tasks exceed 180 (utilization $\rightarrow 95\%$), the SLA violation gaps among all schedulers narrow, with STPS converging to 36.8% at 200 tasks. This validates our analytical model: at extreme physical saturation, the background temporal traffic becomes a densely packed, flat high-water mark. The temporal variance approaches zero, leaving Step B with no valleys to exploit ($\Delta t_{start}^* \rightarrow 0$). At this limit, temporal load balancing naturally degrades, and STPS elegantly falls back to relying purely on its optimized spatial placement (Step A + C), proving its robustness across the entire load spectrum.

7 Conclusion

This paper presents GLaSS, a dynamic scheduler for SNN cloud services. By leveraging accumulated spike activity and ROI-based migration, GLaSS solves the static binding inefficiency inherent in neuromorphic hardware. Future work includes implementing predictive scaling based on historical spike patterns.

Acknowledgments

The USENIX latex style is old and very tired, which is why there's no \acks command for you to use when acknowledging. Sorry.

Availability

USENIX program committees give extra points to submissions that are backed by artifacts that are publicly available. If you made your code or data available, it's worth mentioning this fact in a dedicated section.

References

- [1] Sultan Mahmud Sajal, Luke Marshall, Beibin Li, Shandan Zhou, Abhisek Pan, Konstantina Mellou, Deepak

Narayanan, Timothy Zhu, David Dion, Thomas Moscibroda, and Ishai Menache. Kerveros: Efficient and scalable cloud admission control. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 227–245, Boston, MA, July 2023. USENIX Association.

SNN schedule/picture/architecture.png

Figure 1: The proposed STPS framework for multi-card CIM clusters.

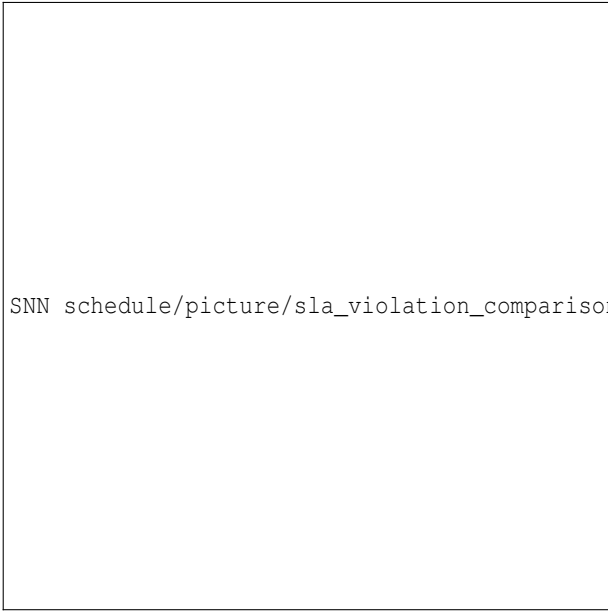


Figure 2: SLA Violation Rate (Instantaneous load $> BW_{max}$) under Bursty Workloads. Baselines fail due to the lack of temporal interleaving.



Figure 3: Impact of Task Density on SLA Violations. STPS unlocks hidden cluster capacity by treating time as an allocatable resource.